

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192525199
Name	G. Venkata Niranjan Reddy

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

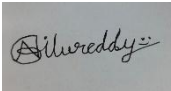
Total Marks: 50

Code of Conduct

I G. Venkata Niranjan Reddy (192525199) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. HDFS-Based Storage Solution for a Media Platform

Problem:

A media platform stores large amounts of videos, images, audio files, and metadata. The system must maintain **more than 99.9% availability**, use a **maximum replication factor of 3**, and keep storage costs efficient.

Proposed Design

Use **HDFS** with the following configuration:

- HDFS NameNode configured with **High Availability (HA)** using Active and Standby NameNodes.
- Data stored across multiple DataNodes.
- Set **replication factor = 3** for important media data.
- Use HDFS block size such as **128 MB or 256 MB** for large media files.
- Use rack-aware placement so replicas are distributed across different racks.
- Add DataNodes as storage grows instead of continuously increasing replication.
- Use compression for metadata and suitable media formats.
- Move rarely accessed data to lower-cost/archive storage when appropriate.

Constraint Satisfaction

1. Availability > 99.9%

NameNode HA prevents a single NameNode failure from making the entire HDFS unavailable. Replicas on multiple DataNodes allow the system to continue serving data when a DataNode fails.

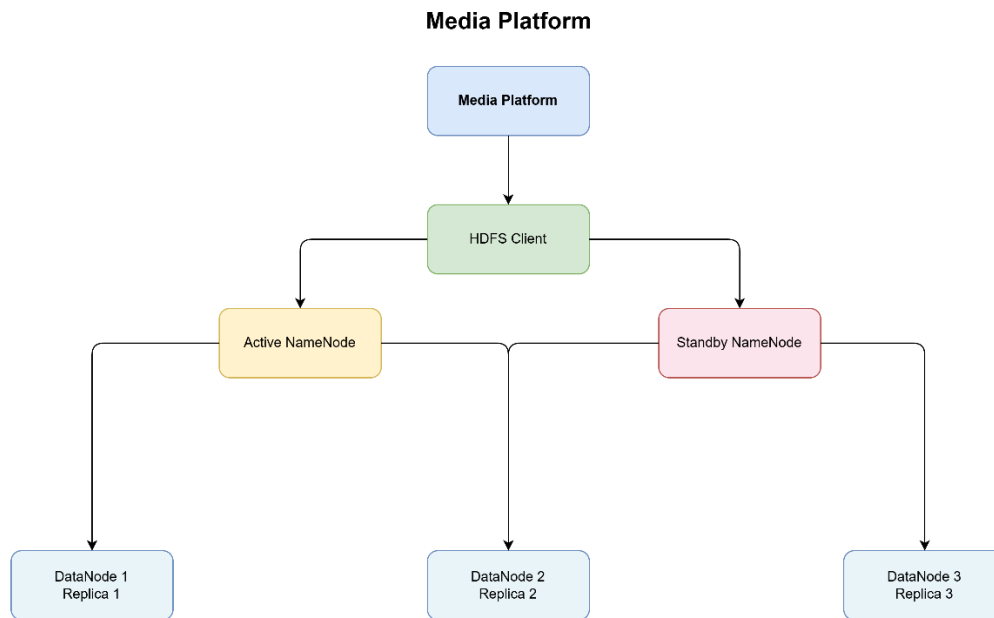
2. Replication ≤ 3

Use a replication factor of exactly **3** for critical data. HDFS stores three copies on different nodes/racks, providing fault tolerance without excessive storage consumption.

3. Cost efficiency

Instead of increasing replication, storage capacity can be expanded by adding DataNodes. Large block sizes reduce metadata overhead, while compression and archival storage reduce storage requirements.

Simple Architecture



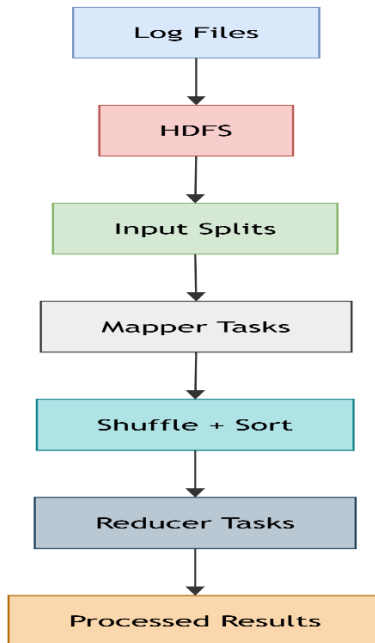
2. MapReduce Workflow Within a 20-Minute Batch Window

Problem:

Log data must be processed within **20 minutes**, but the cluster has a limited number of nodes.

Proposed Job Design

Use a parallel MapReduce design:



Mapper

Each mapper processes a separate input split.

For example, logs may contain:

IP Address | Timestamp | URL | Status Code

The mapper extracts useful information and generates key-value pairs such as:

URL → 1

IP → 1

StatusCode → 1

Shuffle and Sort

MapReduce automatically groups identical keys:

/page1 $\rightarrow$ [1,1,1,1]

/page2 $\rightarrow$ [1,1,1]

Reducer

The reducer aggregates the values:

/page1 $\rightarrow$ 4

/page2 $\rightarrow$ 3

Task Distribution

Since the number of nodes is limited:

- Divide input into appropriately sized HDFS blocks.
- Run multiple mapper tasks in parallel on available nodes.
- Keep reducer count appropriate to cluster capacity.
- Prefer **data locality**, running computation close to the data.
- Avoid unnecessary intermediate data.
- Use combiners to reduce mapper-to-reducer network traffic.
- Allocate sufficient containers/CPU/memory to the MapReduce job.
- Monitor task execution and handle slow or failed tasks.

Meeting the 20-Minute Constraint

The total processing time can be represented as:

Input $\rightarrow$ Map $\rightarrow$ Shuffle/Sort $\rightarrow$ Reduce $\rightarrow$ Output

To meet 20 minutes:

- Increase parallelism within available nodes.
- Use combiners.
- Optimize mapper and reducer code.
- Partition data evenly to avoid reducer bottlenecks.
- Use compressed intermediate data where beneficial.

3. YARN Scheduler Policy

Problem:

A YARN cluster runs:

1. ETL jobs
2. Reporting jobs
3. Machine Learning jobs

The scheduler must provide **fairness**, give **priority to ETL**, and recover from **node failures**.

ETL Priority

ETL receives a larger guaranteed capacity because it is important for preparing data for downstream applications.

For example:

ETL Queue → 50%

Reporting Queue → 25%

ML Queue → 25%

Unused resources can be temporarily allocated to other queues.

Fairness

Within each queue, resources are distributed fairly among jobs. This prevents one large job from consuming all available cluster resources.

Node Failure Recovery

YARN uses the:

- **ResourceManager**
- **NodeManager**
- **ApplicationMaster**

If a NodeManager fails:

1. ResourceManager detects the failure.

2. Containers running on that node are considered lost.
3. ApplicationMaster requests new containers.
4. Failed tasks are restarted on healthy nodes.
5. Processing continues.

Constraint Satisfaction

Fairness: Resources are distributed among queues and jobs.

ETL priority: ETL receives higher queue priority and guaranteed capacity.

Failure recovery: Failed containers/tasks are rescheduled automatically on healthy nodes.

4. Enterprise Data Ingestion from SAN, NAS, and Databases

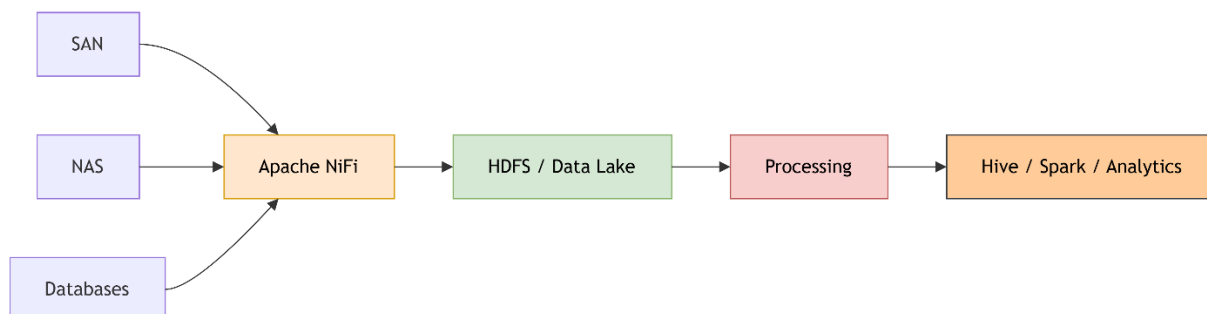
Problem:

An enterprise must ingest data from:

- SAN
- NAS
- Operational databases

while maintaining **backup reliability** and minimizing **duplicate data**.

Proposed Architecture



Components

1. SAN

SAN contains enterprise block-level storage.

Use appropriate connectors or file-access mechanisms to extract required datasets into the ingestion pipeline.

2. NAS

NAS provides shared file-based storage.

NiFi can monitor directories and ingest new or changed files.

3. Operational Databases

Use database connectors and incremental extraction.

Instead of repeatedly copying the entire database, extract only:

- New records
- Updated records
- Required changes

using timestamps, transaction IDs, or CDC mechanisms.

4. Apache NiFi

NiFi acts as the central ingestion layer.

It provides:

- Data routing
- Transformation
- Filtering
- Monitoring
- Back-pressure
- Flow management
- Provenance tracking

5. HDFS/Data Lake

Store ingested data in HDFS using organized directories such as:

/data/raw

/data/processed

/data/archive

Minimizing Duplication

Use:

- Unique record IDs
- Source-system IDs
- Checksums/hashes
- Incremental ingestion
- Deduplication before storing processed data

Backup Reliability

Use:

- HDFS replication
- Periodic backups
- Snapshot/archive policies
- Separate backup storage
- Metadata and configuration backups

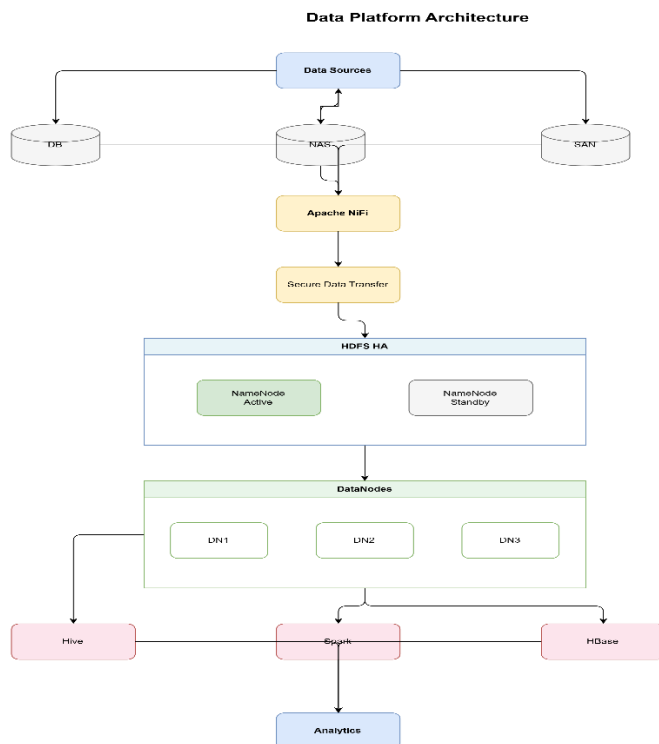
5. End-to-End Hadoop Ecosystem Solution

Problem:

Design a Hadoop ecosystem that supports:

- Secure data movement
- High availability
- Integration with Apache NiFi/ETL pipelines

Proposed Architecture



1. Secure Data Movement

Use security mechanisms such as:

- **Kerberos authentication**
- TLS/SSL encryption
- Access control lists

- Encryption at rest where required
- Secure NiFi connections

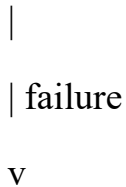
NiFi can authenticate and securely transfer data before writing it into HDFS.

2. High Availability

Configure:

NameNode HA

Active NameNode



Standby NameNode

DataNodes store replicated copies of blocks.

Use replication factor **3** for important datasets.

YARN ResourceManager HA can also be configured to prevent ResourceManager failure from stopping cluster operations.

3. Apache NiFi Integration

NiFi receives data from different sources and performs:

Ingest



Validate



Transform



Filter



Route



Store in HDFS

NiFi also provides flow monitoring and provenance information.

4. Hadoop Processing

After data enters HDFS:

- **Hive** → SQL-based analytics
- **Spark** → fast large-scale processing and ETL
- **MapReduce** → batch processing
- **HBase** → low-latency access to large datasets